

Object-Oriented Programming

Object Oriented Programming? A way of writing code where we group related data and functions into a single unit called a **Class**, and use **Objects** to interact with that data while keeping the internal details hidden.

Class is a blueprint or template. It defines the structure (data) and behavior (functions) of its objects, but it doesn't hold any actual data itself. Think of it like an architectural blueprint for a car. It describes what the car will look like, but it isn't a car.

Object is a concrete instance created from a class. It holds actual data in the attributes defined by the class. Using the same analogy, an object is the actual car built from that blueprint. You can build many cars from the same blueprint, each with different paint colors or engines.

```
class Car {  
    public:  
        string brand;  
        int price;  
        int speed;  
  
        void display() {  
            cout << "Brand: " << brand << endl;  
            cout << "Speed: " << speed << endl;  
            cout << "Price: " << price << endl;  
        }  
};
```

Under the Hood: When you create **1,000** Car objects, **C++** does **not** copy the **display()** function **1,000** times. Memory is allocated only for the variables. The member functions are stored exactly once in the code segment of memory. The compiler secretly passes a hidden **this** pointer to the function so it knows which specific object's data to print.

Access Specifiers

In C++, they control who is allowed to look at or change the variables inside your class. Think of **Access Specifiers** as the security clearance levels for your data.

There are **3** Access specifiers:

- **Public:** Allows access from **anywhere** in the program.
- **Private:** Restricts access **only** to the class that defines it.
- **Protected:** Restricts access to the defining class and its **subclasses**.

Under the Hood: Access control is strictly a **compile-time** illusion. The C++ compiler uses these keywords to throw errors if you break the rules while writing code. However, at **run-time**, the CPU and RAM have absolutely zero concept of private or public. Because of this, you can actually bypass private access restrictions in C++ by using raw pointer arithmetic to jump directly to the memory address of the private variable.

“This” Pointer

The **“this”** pointer is a hidden pointer available as a parameter inside all non-static functions of a class. It holds the specific memory address of the object that calls the function

It allows:

- **Identifies the Caller:** It distinguishes between class attributes and local parameters when they have the same name.

```
void setSpeed(int speed) {  
    this->speed = speed;  
}
```

- **Enables Chaining:** It allows methods to return a reference to the current object, so multiple functions can be called in a single line.

```
Car& setBrand(string brand) {  
    this->brand = brand;  
    return *this;  
}  
// Usage: myCar.setBrand("Toyota").setSpeed(100);
```

- **Passes Itself:** It allows an object to pass its own memory address as an argument to other functions or classes.

```
void requestService(Mechanic& m) {  
    m.inspect(this);  
}
```